

Wallet Activity API — Reference & QA Test Guide

1. What this API is, in plain terms

Every partner that uses our platform has a **wallet** — a running balance of money they've loaded, plus a credit line, that gets debited when they pay for a booking and credited when they top up, get a refund, or earn a reward.

This feature gives a partner **visibility into that wallet**, the same way a bank gives you a statement and lets you tap a transaction to see more:

Everyday equivalent	This API
Your bank statement — the list of transactions and your current balance	Endpoint 1: the activity feed
Tapping one transaction to see the full breakdown	Endpoint 2: the activity detail

Both are **read-only** — calling them cannot move money, change a balance, or alter anything. They only report what has already happened.

Under the hood, the actual wallet balances and transaction history live in a separate internal system (we refer to it as "the wallet service" or "Paynex" in this document). These two APIs are a **wrapper** around it: they fetch the raw data, translate it into plain, stable, partner-facing language, and deliberately hide internal plumbing (database ids, internal codes, dedup keys) that a partner has no use for and should never see. That's a design choice, not an oversight — see §5 before treating a "missing" field as a bug.

2. The two endpoints, at a glance

Method	Route	What it gives you
GET	<code>/v1/wallet/activity</code>	A page of the wallet's recent money movements, newest first, plus the current balance.
GET	<code>/v1/wallet/activity/{id}</code>	The full story on one movement from that list — in particular, if it involved money being set aside for a booking, exactly how that played out.

How they relate: every row in the feed has an `id`. Take that `id` and call the second endpoint to drill into it. You always start at the feed; the detail endpoint is for expanding one row you already saw there.

3. Key terms (read this before the test cases)

Term	What it means here
Wallet	The partner's running account: cash balance + credit line.

Term	What it means here
Movement / activity entry	Anything that changed, or is about to change, what the partner can spend — a top-up, a booking payment, a refund, etc. One row in the feed.
Hold	Money set aside , not yet spent, not yet given back — like a hotel putting a "pending" charge on your card before checkout. Placed when a booking starts, and resolved one of three ways: paid out to the booking, given back because the booking was cancelled, or auto-returned because it timed out unused.
Category	A plain label for what kind of movement a row is — see the full list in §6.
Kind	The direction of money: credit (added to the wallet), debit (taken out), or hold (set aside, not yet gone).
Reference	A short human-readable pointer to <i>what this movement was about</i> — a bank transaction id for a top-up, or your own booking reference for anything tied to a booking.
Currency / amount	Always shown as ordinary decimal rupees (4500.00 , not 450000) with the currency code alongside every amount.

The "hold" story, illustrated

A flight gets repriced after booking. ₹6,000 was originally set aside; the final fare is ₹5,800, so ₹200 goes back to the wallet.

1. **Hold placed** — ₹6,000 set aside. Feed shows one row: category **funds_held**, kind **hold**. The partner cannot spend that ₹6,000, but it hasn't left the wallet either.
2. **Booking confirmed** — ₹5,800 is paid out, ₹200 is released back. The **funds_held** row **disappears** from the feed and is replaced by two rows: a **booking_payment** (debit, ₹5,800) and a **hold_released** (credit, ₹200).

This is deliberate, not a bug: once money is no longer "set aside," the feed tells the story of where it actually went instead of also showing the hold that led there — so the same money is never counted twice. Calling the detail endpoint on either of those two settlement rows gives back the full picture — held, paid out, released, and the timeline of all three steps — as one object.

4. Before you start testing

1. **base_url** — the API host for the environment under test.
2. **access_token** — a partner's bearer token (from **POST /v1/auth/partner/token**). This is the **only** credential either endpoint needs — no API key, no request signature, no idempotency key. (If you've tested the wallet top-up endpoint, that one needs several more headers because it moves money; these two only read, so one token is enough. That difference is intentional, not a gap in this test pass.)
3. The test partner must have a wallet actually assigned to them (**wallet_owner_id** set) and the wallet integration must be switched on in the environment (**b2b_wallet.enabled: true**). Otherwise every call correctly reports an error — see §9 — rather than a result.
4. **Test data on the wallet**. Ideally the wallet has, before you start:

- at least one **top-up**
- at least one **fully paid booking** (a settled, committed hold)
- at least one **cancelled or expired** booking (a released/expired hold)
- if achievable, one booking **in progress right now** (a currently outstanding hold) — see §10 before spending much time chasing this specifically.

5. Things that are missing on purpose

The internal wallet service returns more fields than this API exposes. If you compare against the internal contract and a field looks "missing," check this list before filing it:

What's hidden	Why
Internal dedup keys, internal category codes, internal reference-type tags	Plumbing used to keep the ledger consistent internally. Meaningless to a partner and would leak how the ledger works if exposed.
A separate booking-reference field	Merged into the one reference field described above — there's no reason for a partner to track two reference fields when only one is ever relevant to a given row.
A status field on a hold's detail	The type field on a hold (hold / committed / released / expired) already tells you its state — a second status field next to it would just be a second place for the two to disagree.
Internal wallet/account ids	Never exposed. The partner's access token is the only identifier they need or get.
The wallet service's own error messages, on failure	If the internal service errors out, the partner gets one of <i>this</i> API's own error messages (see §9), never the internal system's raw error text.

None of the above is a defect if it's absent. It becomes a defect only if one of these **does** show up in a response — see the "field curation" test cases below (AF-17, AD-12).

6. Endpoint 1 — Activity feed

```
GET /v1/wallet/activity
```

Request

Headers	Authorization: Bearer {access_token} — required. X-Request-Id , X-Correlation-ID — optional, echoed back on errors.
Query	page — optional, 1-based, defaults to 1 . Page size is fixed at 15 and cannot be changed by the caller.

Example response — 200 OK

```
{
  "summary": {
    "currency": "INR",
    "cashBalance": 11486.00,
    "reserved": 0.00,
    "creditAvailable": 0.00,
    "available": 11486.00
  },
  "pagination": { "page": 1, "perPage": 15, "total": 4, "totalPages": 1 },
  "activity": [
    {
      "id": "3f1c88d0-5a21-4a7e-9f0b-2c4e6a1d9b73",
      "category": "funds_held",
      "kind": "hold",
      "amount": 4500.00,
      "currency": "INR",
      "status": "COMPLETED",
      "reference": "BR1787400000000001",
      "date": "2026-08-21T12:02:11.004Z"
    }
  ]
}
```

Field reference

summary — the wallet's balance *right now*. This does not change as you page through history; it's always today's numbers.

Field	Meaning
currency	e.g. INR.
cashBalance	Funds immediately available to spend.
reserved	Currently tied up in holds.
creditAvailable	Remaining credit line.
available	cashBalance + creditAvailable.

pagination

Field	Meaning
page	The page you requested.
perPage	Always 15.
total	Total movements across all pages.
totalPages	How many pages there are in total.

activity[] — one row per movement, newest first.

Field	Meaning
id	Pass this to the detail endpoint to see more.
category	What kind of movement this is — table below.
kind	credit , debit , or hold (see §3).
amount	Decimal rupees.
currency	e.g. INR .
status	PENDING COMPLETED FAILED REVERSED .
reference	What the movement is about — a bank/gateway reference, or a booking reference. null if neither applies.
date	When it happened.

category values

Value	Plain meaning
top_up	Partner added money.
booking_payment	Money paid out for a confirmed booking.
credit_drawn	Funds drawn from the credit line.
credit_repaid	Credit line paid back.
refund	A refund credited back.
adjustment	A manual correction to the balance.
funds_held	Money currently set aside for a booking in progress (see §3).
hold_released	A hold given back, unused (or partly unused).
hold_expired	A hold that timed out before it was used.
reward	A reward credited to the wallet.
promotion	A promotional credit.
redemption	A reward or promotional credit spent.

Test cases

ID	Scenario	Steps	Expected result
AF-01	Happy path, page 1	Call the endpoint with a valid token	200 ; summary , pagination , activity all present; pagination.perPage === 15
AF-02	Page 2+	Repeat with ?page=2 while total > 15	200 ; different rows than page 1; no row repeated across pages; no row skipped

ID	Scenario	Steps	Expected result
AF-03	Empty wallet	Use a partner with no activity yet	<code>200; activity: []; pagination.total === 0; summary</code> is still a real balance (zero, not an error)
AF-04	<code>page</code> omitted	Call with no query string	Same as page 1
AF-05	<code>page=0</code>	<code>?page=0</code>	<code>400, error.code: VALIDATION_ERROR</code> , message mentions "page must be 1 or greater"
AF-06	Non-numeric <code>page</code>	<code>?page=abc</code>	<code>400, VALIDATION_ERROR</code>
AF-07	Decimal <code>page</code>	<code>?page=1.5</code>	<code>400, VALIDATION_ERROR</code>
AF-08	Negative <code>page</code>	<code>?page=-1</code>	<code>400, VALIDATION_ERROR</code>
AF-09	Absurdly large <code>page</code>	<code>?page=999999999</code>	<code>400, VALIDATION_ERROR</code>
AF-10	No Authorization header	Omit it entirely	<code>401</code> ; response body contains no wallet data at all (no balance, nothing)
AF-11	Malformed Authorization header	<code>Authorization: Bearer</code> with nothing after it, or the wrong scheme	<code>401, error.code: MISSING_AUTHORIZATION_HEADER</code>
AF-12	Garbage token	A random string as the token	<code>401, error.code: INVALID_ACCESS_TOKEN</code>
AF-13	Revoked token	A token that was explicitly revoked	<code>401, error.code: TOKEN_REVOKED</code>
AF-14	Expired token	A token past its expiry	<code>401, error.code: ACCESS_TOKEN_EXPIRED</code>
AF-15	Partner with no wallet	A partner not yet given a wallet	<code>404, error.code: WALLET_NOT_CONFIGURED</code>
AF-16	Wallet integration switched off	Environment has it disabled	<code>503, error.code: WALLET_DISABLED</code>
AF-17	No internal data leaks	Inspect the raw response text, not just the fields you expect	None of the internal-only names from §5 appear anywhere, at any nesting level
AF-18	Category / kind are always from	Look at every row in a full feed	<code>category</code> is always one of the values in the table above; <code>kind</code> is <code>credit</code> , <code>debit</code> , <code>hold</code> , or

ID	Scenario	Steps	Expected result
	the published lists		<code>null</code> — never an internal code
AF-19	Hold rows are self-consistent	Look at every row	Every row with <code>category: funds_held</code> has <code>kind: hold</code> , and every row with <code>kind: hold</code> has <code>category: funds_held</code> — they must always agree
AF-20	Reference falls back to the booking	Find a row settling a hold (e.g. <code>hold_released</code>)	<code>reference</code> shows the booking reference, even though that specific row may have no bank/gateway reference of its own
AF-21	Oversized request-id header	Send a 129+ character <code>X-Request-Id</code>	<code>400, VALIDATION_ERROR</code>
AF-22	Request id is echoed	Send <code>X-Request-Id: qa-test-1</code>	On an error response, <code>error.request_id</code> equals <code>"qa-test-1"</code>

7. Endpoint 2 — Activity detail

```
GET /v1/wallet/activity/{id}
```

Take any `id` from the feed and call this to see the full story behind it.

Request

Headers	Same as endpoint 1.
Path	<code>{id}</code> — an <code>id</code> from the feed.

Example response — a hold that was paid out

```
{
  "type": "committed",
  "id": "d62def98-3aa9-4d96-ab80-6d11a6744858",
  "reference": "BR1787311485247078",
  "currency": "INR",
  "held": 6000.00,
  "committed": 5800.00,
  "released": 200.00,
  "stillHeld": 0.00,
  "createdAt": "2026-08-21T11:24:45.118Z",
  "expiresAt": "2026-08-21T11:39:45.118Z",
  "legs": [
    { "label": "Held for booking", "kind": "hold", "amount": 6000.00,
```

```

"date": "...",
  { "label": "Committed to booking", "kind": "debit", "amount": 5800.00,
"date": "...",
  { "label": "Released back to your balance", "kind": "credit", "amount":
200.00, "date": "..."}
]
}

```

Example response — a standalone movement (top-up, refund, etc.)

```

{
  "type": "transaction",
  "id": "75ee1862-de5e-43d5-9e06-bd8f5df5d7d0",
  "category": "top_up",
  "kind": "credit",
  "amount": 10000.00,
  "currency": "INR",
  "status": "COMPLETED",
  "reference": "PAY-1781603105422",
  "date": "2026-07-02T08:06:27.466Z"
}

```

Which shape you get back

The **type** field tells you which of the two shapes above you're looking at:

type	Meaning	Has legs?
hold	Still set aside — nothing paid out or returned yet	Yes
committed	Paid out to the booking	Yes
released	Given back, unused	Yes
expired	Timed out and auto-returned	Yes
transaction	Anything else (top-up, refund, credit draw, adjustment, ...)	No

Hold fields

Field	Meaning
type	The state — see table above.
id	The hold's own id.
reference	Your booking reference.
currency	e.g. INR .
held	Original amount set aside.

Field	Meaning
<code>committed</code>	How much was actually paid to the booking.
<code>released</code>	How much came back to the wallet.
<code>stillHeld</code>	How much is still set aside right now. Always: <code>held = committed + released + stillHeld</code> .
<code>createdAt</code>	When the hold was placed.
<code>expiresAt</code>	When it will auto-return if nothing happens first.
<code>legs[]</code>	The step-by-step story: <code>label</code> (plain description), <code>kind</code> , <code>amount</code> , <code>date</code> .

Transaction fields

Same as a feed row (`category`, `kind`, `amount`, `currency`, `status`, `reference`, `date`) — no `legs`, because there's nothing to break down.

Two things to test carefully, because they're easy to get wrong:

- A **hold** response never has a `status` field — its `type` already tells you the state. If one appears, that's a defect.
- If the wallet service ever reports a hold state not in the table above, it should still come through as **that exact word**, lowercased — not silently changed to one of the four known values. Don't assume a fifth value appearing is a bug; check whether it was passed through faithfully before reporting it.

Test cases

ID	Scenario	Steps	Expected result
AD-01	Detail of a top-up	Take the <code>id</code> of a <code>top_up</code> feed row, look it up	<code>200</code> ; <code>type: "transaction"</code> ; <code>category: "top_up"</code> ; no <code>legs</code> key
AD-02	Detail of a paid-out hold	Take the <code>id</code> of a <code>booking_payment</code> feed row, look it up	<code>200</code> ; <code>type: "committed"</code> ; <code>legs</code> present (typically 3 steps); <code>held = committed + released + stillHeld</code> . If this instead comes back as <code>type: "transaction"</code> , note it as a finding rather than assuming the test steps are wrong
AD-03	Detail of a released hold	Same, for a cancelled booking	<code>200</code> ; <code>type: "released"</code> ; <code>legs</code> present
AD-04	Detail of an expired hold	Same, for a lapsed booking	<code>200</code> ; <code>type: "expired"</code> ; <code>legs</code> present
AD-05	Detail of a hold in progress	Same, while a booking is still being held	<code>200</code> ; <code>type: "hold"</code> ; <code>stillHeld > 0</code> ; <code>committed</code> and <code>released</code> are <code>0</code>

ID	Scenario	Steps	Expected result
AD-06	Leg shape	Look at <code>legs[]</code> on any hold	Each step has only <code>label</code> , <code>kind</code> , <code>amount</code> , <code>date</code> — nothing else
AD-07	Unknown id	A well-formed but made-up id	<code>404, error.code: ACTIVITY_NOT_FOUND</code>
AD-08	Someone else's id	Using partner A's token, look up an id that belongs to partner B	<code>404</code> — identical to AD-07 in every way. It must be impossible to tell "this id doesn't exist" apart from "this id exists but isn't yours"
AD-09	Malformed id	Try things like <code>../balance</code> , <code>a/b</code> , or an extremely long string in place of the id	<code>400, error.code: VALIDATION_ERROR</code>
AD-10	Missing id	Call the endpoint with nothing after the trailing slash	Should be rejected outright, not treated as a blank id
AD-11	No Authorization header	Omit it	<code>401</code> ; nothing about whether the id exists is revealed
AD-12	No internal data leaks	Inspect the raw response for both a hold and a transaction	Same check as AF-17, plus: <code>status</code> must be absent on a hold specifically
AD-13	Partner with no wallet	Partner has no wallet assigned	<code>404, error.code: WALLET_NOT_CONFIGURED</code> — same status as AD-07/AD-08 but a different code
AD-14	Wallet integration switched off	Environment has it disabled	<code>503, error.code: WALLET_DISABLED</code>
AD-15	Reference on a transaction	Detail of a top-up	<code>reference</code> is the bank/gateway reference; never a booking reference (a standalone movement has no booking)
AD-16	Oversized request-id header	129+ characters	<code>400, VALIDATION_ERROR</code>

8. Checks that apply to both endpoints

- **No internal data ever leaks.** Nothing from §5 appears anywhere in a response — including inside error messages.
- **Failures never expose the internal system's own wording.** If the underlying wallet service is down or errors out, the message the partner sees is always one of *this* API's own messages (§9),

never the internal system's raw text.

- **Amounts are always decimal rupees with a currency label**, consistently, everywhere — the balance, every feed row, every detail response.
- **Calling either endpoint repeatedly changes nothing**. Same input, same answer, every time (aside from new activity genuinely happening on the wallet in between).

9. Error reference

HTTP status	<code>error.code</code>	What happened
400	<code>VALIDATION_ERROR</code>	A bad <code>page</code> , a malformed id, or an oversized request-id header
401	<code>MISSING_AUTHORIZATION_HEADER</code>	No (or badly formed) Authorization header
401	<code>INVALID_ACCESS_TOKEN</code>	The token isn't in the right format at all
401	<code>TOKEN_REVOKED</code>	The token doesn't exist or was revoked
401	<code>ACCESS_TOKEN_EXPIRED</code>	The token has expired
404	<code>WALLET_NOT_CONFIGURED</code>	This partner has no wallet assigned
404	<code>ACTIVITY_NOT_FOUND</code>	The requested id doesn't exist, or isn't this partner's (detail endpoint only)
503	<code>WALLET_DISABLED</code>	The wallet integration is switched off
502	<code>WALLET_ERROR</code>	The internal wallet service answered with an error, or its response couldn't be read
503	<code>WALLET_UNAVAILABLE</code>	The internal wallet service didn't respond at all
500	<code>INTERNAL_ERROR</code>	Something failed on our own side, unrelated to the above

Every error looks like this:

```
{
  "error": {
    "code": "...",
    "message": "...",
    "details": "...",
    "timestamp": "2026-08-26T12:00:00Z",
    "request_id": "...",
    "correlation_id": "..."
  }
}
```

`request_id` echoes back whatever you sent in `X-Request-Id` (or `null` if you didn't send one). `correlation_id` is always present, generated automatically if you don't supply one.

10. Known gaps

Read this before filing something below as a bug — it may already be a known, tracked limitation rather than a surprise.

1. **A hold currently in progress hasn't been proven end-to-end on live data.** Test data with a booking actively "in hold" right now is hard to come by, on both sides of this integration. If you're able to set one up, AF-19 and AD-05 are the tests that matter most — that's the highest-value thing to verify if you can.
 2. **Only rupee (INR) wallets have been exercised.** The rupee conversion logic assumes 100 minor units per major unit, which is correct for INR but hasn't been tested against a different currency/asset type, since no test wallet holds one today. Not something to chase actively — just flag it if a non-INR wallet ever turns up in testing.
 3. **Filtering by date range or movement type isn't available yet**, even though the underlying system supports it. `page` is the only thing you can pass in the query string today; anything else you add is silently ignored rather than rejected.
-